ATLAS / ASSESSMENT SUBMISSION

Your work, in motion.

DRAFT — replace placeholders and verify the deployed app before submission.

Candidate: REPLACE_FIRSTNAME REPLACE_LASTNAME
Live demo: REPLACE_LIVE_DEMO_URL
GitHub repository: REPLACE_GITHUB_REPOSITORY_URL
Final filename: firstname_lastname_task_manager_assessment.pdf

Overview and design decisions

Atlas is a responsive, private task board built with React, TypeScript and Vite. Supabase supplies anonymous authentication and Postgres persistence. GitHub hosts the source and Vercel's free tier is the intended deployment target. A separate backend is unnecessary because Supabase provides a client API protected by row-level security.

The cream and sage palette, consistent typography, compact cards and four clear status columns create a calm workspace. Empty states explain what to do next; loading, failure and success messages make state visible. Task dialogs support keyboard focus containment and status changes without dragging. On small screens, columns scroll horizontally.

Core functionality

Visitors automatically receive a Supabase anonymous identity, reused in the same browser. Tasks contain a required title, optional description and due date, priority, status, creation timestamp, owner and arrays of assignee/label UUIDs. Tasks move between To Do, In Progress, In Review and Done. Successful drops persist to Supabase; failed drops preserve the previous state. The editor supports creating, updating and deleting tasks.

Advanced features implemented

1. Team profiles with names, colors and initials; multiple assignees per task. Profiles organize a single guest's work and do not represent invited login accounts.
2. Comments stored separately, shown in chronological order with timestamps.
3. Server-generated immutable activity records for creation, status changes, content edits, priority, due dates, assignments and labels.
4. Custom colored labels, multiple labels per task, label filter.
5. Overdue and due-soon card indicators using local calendar dates; completed tasks excluded from overdue counts.
6. Case-insensitive title search plus priority, assignee and label filters combined with AND.
7. Total, completed, overdue and completion-percentage summary for the full board.

Security and data integrity

RLS is enabled on all five tables and scoped to the authenticated guest's auth.uid(). Child inserts validate task ownership. Task triggers reject foreign or duplicate member/label references and prevent ownership changes. Audit writes occur only through a narrowly scoped server trigger; clients can read their own audit records but cannot forge them. Foreign keys cascade comments and activity when a task is deleted. Only the public browser key is used; no service-role key belongs in the frontend. updated_at checks reject stale task edits instead of silently overwriting changes.

Local setup

1. Install Node.js 22.12+ or Node 24 LTS and Git; clone the repository.
2. Run npm ci in the project folder.
3. Create a free Supabase project and enable Anonymous Sign-Ins.
4. Run the complete SQL schema in supabase/schema.sql once in a fresh project.
5. Copy .env.example to .env. Set VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY to that project's public values.
6. Run npm run dev and open the printed local URL.
7. Run npm test and npm run build. For a production preview, run npm run preview.
8. Deploy to Vercel with Vite preset, build command npm run build, output dist, and both public environment variables.

Verification

Completed locally: TypeScript/production build; nine domain tests covering filtering and local calendar urgency. The SQL schema executes in embedded PostgreSQL, where guest isolation, child ownership, audit triggers, immutable task ownership and deletion cascades pass checks. These checks do not substitute for a real deployed Supabase test.

REPLACE_LIVE_TEST_RESULTS: run the two-guest API script and manual acceptance checklist in docs/TESTING.md; record actual results and date here. Browser UI verification details are recorded separately in docs/VERIFICATION.md.

Tradeoffs and next improvements

Saves occur on drop; remote tabs refresh manually rather than through realtime subscriptions. Member and label catalogs are creation-only, and bounded UUID arrays simplify multiple assignments; normalized join tables and catalog lifecycle controls would suit a larger app. Card order is creation order within each column. Anonymous browser identities cannot be recovered after browser storage is cleared. Comments are append-only. The app does not queue offline changes or paginate large boards. Add guest-to-permanent account conversion, abuse protection/CAPTCHA integration, realtime updates, pagination, normalized joins and broader automated browser coverage for production use.

Full database schema

Exact setup SQL. Run once in a fresh Supabase project.

```
-- Run ONCE in a fresh Supabase project's SQL Editor. No service key required.
begin;
create table public.team_members (
 id uuid primary key default gen_random_uuid(), user_id uuid not null default auth.uid() references
auth.users(id) on delete cascade,
 name text not null check (length(btrim(name)) between 1 and 80),
 color text not null default '#6c7053' check (color ~ '^#[0-9a-fA-F]{6}$'),
 created_at timestamptz not null default now(), unique(user_id,name)
);
create table public.labels (
 id uuid primary key default gen_random_uuid(), user_id uuid not null default auth.uid() references
auth.users(id) on delete cascade,
 name text not null check (length(btrim(name)) between 1 and 40),
 color text not null default '#6c7053' check (color ~ '^#[0-9a-fA-F]{6}$'),
 created_at timestamptz not null default now(), unique(user_id,name)
);
create table public.tasks (
 id uuid primary key default gen_random_uuid(), user_id uuid not null default auth.uid() references
auth.users(id) on delete cascade,
 title text not null check (length(btrim(title)) between 1 and 160),
 description text not null default '' check (length(description)<=10000),
 status text not null default 'todo' check (status in ('todo','in_progress','in_review','done')),
 priority text not null default 'normal' check (priority in ('low','normal','high')),
 due_date date,
 assignee_ids uuid[] not null default '{}' check (cardinality(assignee_ids)<=20),
 label_ids uuid[] not null default '{}' check (cardinality(label_ids)<=20),
 created_at timestamptz not null default now(), updated_at timestamptz not null default
clock_timestamp()
);
create table public.comments (
 id uuid primary key default gen_random_uuid(), user_id uuid not null default auth.uid() references
auth.users(id) on delete cascade,
 task_id uuid not null references public.tasks(id) on delete cascade,
 body text not null check (length(btrim(body)) between 1 and 2000), created_at timestamptz not null
default now()
);
create table public.task_activity (
 id uuid primary key default gen_random_uuid(), user_id uuid not null references auth.users(id) on
delete cascade,
 task_id uuid not null references public.tasks(id) on delete cascade,
 message text not null, created_at timestamptz not null default clock_timestamp()
);
create index tasks_owner on public.tasks(user_id,created_at);
create index members_owner on public.team_members(user_id);
create index labels_owner on public.labels(user_id);
create index comments_task on public.comments(task_id,created_at);
create index activity_task on public.task_activity(task_id,created_at);
create index comments_owner on public.comments(user_id);
create index activity_owner on public.task_activity(user_id);
-- All child references must belong to the same guest. Validation applies even
-- to direct REST requests, not just the UI. Arrays allow multiple assignees.
create function public.validate_task() returns trigger language plpgsql set search_path='' as $$
begin
 if TG_OP='UPDATE' then
  if new.user_id<>old.user_id or new.id<>old.id then raise exception 'Task ownership and identity are
immutable'; end if;
  new.created_at:=old.created_at;
 end if;
 new.title:=btrim(new.title);
 if exists(select 1 from unnest(new.assignee_ids) as a(id) where a.id is null or not exists(select 1
from public.team_members m where m.id=a.id and m.user_id=new.user_id)) then raise exception 'Invalid
assignee'; end if;
 if exists(select 1 from unnest(new.label_ids) as a(id) where a.id is null or not exists(select 1 from
public.labels l where l.id=a.id and l.user_id=new.user_id)) then raise exception 'Invalid label'; end
if;
 if cardinality(new.assignee_ids)<>(select count(distinct id) from unnest(new.assignee_ids) as a(id))
or cardinality(new.label_ids)<>(select count(distinct id) from unnest(new.label_ids) as a(id)) then
```

```
raise exception 'Duplicate assignments or labels'; end if;
 new.updated_at:=clock_timestamp();
 return new;
end $$;
create trigger validate_task before insert or update on public.tasks for each row execute function
public.validate_task();
-- Restricted definer function writes immutable, server-generated audit records.
-- No dynamic SQL; fixed empty search_path; cannot be called as a normal RPC.
create function public.audit_task() returns trigger language plpgsql security definer set
search_path='' as $$
declare changes text[]:='{}';
begin
 if TG_OP='INSERT' then
  insert into public.task_activity(user_id,task_id,message) values(new.user_id,new.id,'Created task');
 else
  if new.status is distinct from old.status then changes:=array_append(changes,'Moved from
'||replace(old.status,'_',' ')||' -> '||replace(new.status,'_',' ')); end if;
  if new.title is distinct from old.title then changes:=array_append(changes,'Updated title'); end if;
  if new.description is distinct from old.description then changes:=array_append(changes,'Updated
description'); end if;
  if new.priority is distinct from old.priority then changes:=array_append(changes,'Changed priority to
 '||new.priority); end if;
  if new.due_date is distinct from old.due_date then changes:=array_append(changes,'Updated due date');
 end if;
  if new.assignee_ids is distinct from old.assignee_ids then changes:=array_append(changes,'Updated
assignees'); end if;
  if new.label_ids is distinct from old.label_ids then changes:=array_append(changes,'Updated labels');
 end if;
  if cardinality(changes)>0 then insert into public.task_activity(user_id,task_id,message)
values(new.user_id,new.id,array_to_string(changes,' · ')); end if;
 end if;
 return new;
end $$;
revoke all on function public.audit_task() from public,anon,authenticated;
revoke all on function public.validate_task() from public,anon,authenticated;
create trigger audit_task after insert or update on public.tasks for each row execute function
public.audit_task();
alter table public.tasks enable row level security;
alter table public.team_members enable row level security;
alter table public.labels enable row level security;
alter table public.comments enable row level security;
alter table public.task_activity enable row level security;
create policy tasks_select on public.tasks for select to authenticated using ((select
auth.uid())=user_id);
create policy tasks_insert on public.tasks for insert to authenticated with check ((select
auth.uid())=user_id);
create policy tasks_update on public.tasks for update to authenticated using ((select
auth.uid())=user_id) with check ((select auth.uid())=user_id);
create policy tasks_delete on public.tasks for delete to authenticated using ((select
auth.uid())=user_id);
create policy members_select on public.team_members for select to authenticated using ((select
auth.uid())=user_id);
create policy members_insert on public.team_members for insert to authenticated with check ((select
auth.uid())=user_id);
create policy labels_select on public.labels for select to authenticated using ((select
auth.uid())=user_id);
create policy labels_insert on public.labels for insert to authenticated with check ((select
auth.uid())=user_id);
create policy comments_select on public.comments for select to authenticated using ((select
auth.uid())=user_id);
create policy comments_insert on public.comments for insert to authenticated with check (
 (select auth.uid())=user_id and exists(select 1 from public.tasks t where t.id=task_id and
t.user_id=(select auth.uid()))
);
create policy activity_select on public.task_activity for select to authenticated using ((select
auth.uid())=user_id);
-- Explicit privileges: deny unauthenticated access and forged audit writes.
revoke all on public.tasks,public.team_members,public.labels,public.comments,public.task_activity from
anon,authenticated;
grant select,insert,update,delete on public.tasks to authenticated;
grant select,insert on public.team_members,public.labels,public.comments to authenticated;
grant select on public.task_activity to authenticated;
commit;
```